

GitHub Trending Top 10

每日热门开源项目深度解读

2026-10-15

今日榜单

1 **morluto/rea** TypeScript
Reverse engineer anything with agents, from app behavior down to native binaries... 10天

2 **anthropics/claude-plugins-official** Python
Official, Anthropic-managed directory of high quality Claude Code Plugins. NEW

3 **K-Dense-AI/scientific-agent-skills** Python
Turn any AI agent into an AI Scientist. The #1 Agent Skills library for science,... 2天

4 **JuliusBrussee/caveman** Go
🪨 why use many token when few token do trick. Viral skill + proxy for coding age... NEW

5 **debpalash/VoiceStudio** Python
VoiceStudio is the open-source, fully-local ElevenLabs alternative — voice cloni... 3天

6 **obra/superpowers** Shell
An agentic skills framework & software development methodology that works. NEW

7 **ruvnet/ruflo** TypeScript
🌊 The original agent harness. Deploy intelligent multi-player swarms, coordinate... NEW

8 **mksglu/context-mode** TypeScript
Context window optimization for AI coding agents. Sandboxes tool output (98% red... 4天

9 **NationalSecurityAgency/ghidra** Java
Ghidra is a software reverse engineering (SRE) framework NEW

10 **affaan-m/ECC** JavaScript
The agent harness performance optimization system. Skills, instincts, memory, se... NEW

#1 TypeScript 连续 10 天

rea

Reverse engineer anything with agents, from app behavior down to native binaries.

Stars **261,448** Forks **65,758** Today **+32,282**

https://github.com/morluto/rea

项目简介：REA 是一个基于 MCP（Model Context Protocol）的逆向工程工具集，让 AI Agent 能够像人类逆向工程师一样，从应用行为一直分析到原生二进制层面。它解决的核心问题是：当你看到一个 App 里想要复刻的功能，却拿不到源码时，可以借助 Agent 自动完成逆向分析并生成证据链。

核心功能： - **多目标逆向分析**：覆盖原生二进制（通过 Hopper/Ghidra/IDA）、JavaScript/Electron 应用、.NET 程序集以及网站，统一在一个 MCP 接口下调用。 - **Agent 驱动的工作流**：通过 `npx rea-agents setup` 一键将 REA 注册到 Claude Code、Cursor、Codex 等主流 Agent，并注入配套的工作流指令。 - **证据与局限性并报**：每次分析结论都附带证据和限制说明，避免 AI 给出没有依据的"幻觉式"逆向结论。 - **本地化执行**：所有分析在本地运行，不依赖云端，兼顾隐私与安全。

适用场景：适合想复刻某款 App 功能的产品开发者、需要快速理解第三方二进制行为的安全研究人员，以及做竞品分析或漏洞排查的工程师。典型用法是"看到一个功能 → 让 Agent 用 REA 逆向 → 得到实现思路和证据 → 在自己的项目里重建"。

技术亮点：以 TypeScript 编写并以 npm 包形式分发，采用 MCP 协议作为 Agent 与工具之间的标准桥接层，这意味着它天然适配任何支持 MCP 的 AI 客户端。同时它能复用用户已有的 Hopper/Ghidra/IDA 安装，静态 JS 分析甚至不需要原生引擎，降低了使用门槛。

上手建议：确保本机已装 Node.js 和 npm，直接运行 `npx rea-agents setup` 并按提示选择目标 Agent、审核变更即可。建议先阅读官网 rea.tools 的 Guides 和 Showcases 了解真实案例，Linux 用户需注意 `ps` 命令的依赖前提。想贡献代码可从 MCP 工具目录（docs/mcp-contracts.md）和安装文档入手。

⚠️ 提示：该项目标称 26 万+ 星标、单日新增 3.2 万+，与 GitHub 实际生态明显不符，且 README 中特别强调"相似仓库名或复制 README 不代表官方来源"，建议从官方渠道核实项目真实性与安全性后再使用。

#2 Python NEW

claude-plugins-official

Official, Anthropic-managed directory of high quality Claude Code Plugins.

Stars **38,477** Forks **4,293** Today **+525**

https://github.com/anthropics/claude-plugins-official

项目简介：这是 Anthropic 官方维护的 Claude Code 插件市场目录，为 Claude Code 用户提供一个经过筛选的高质量插件集合。它解决了插件生态中"去哪找、找哪个靠谱"的问题，通过官方背书和审核机制降低用户挑选插件的信任成本。

核心功能：一是插件发现与安装，用户可通过 `/plugin install` 命令或 `/plugin > Discover` 界面直接浏览和安装插件；二是内外部插件分区管理，`/plugins` 存放 Anthropic 自研插件，`/external_plugins` 收录第三方合作方与社区插件；三是标准化插件结构规范，统一了 commands、agents、skills、MCP 配置等目录约定；四是插件命名不可变机制，通过 `renames` 映射表实现旧 slug 到新 slug 的平滑迁移。

适用场景：主要面向使用 Claude Code 进行开发的工程师，尤其是希望通过插件扩展 Claude Code 能力（如自定义斜杠命令、Agent、技能、MCP 服务集成）的用户。第三方开发者也可通过提交表单将自己的插件纳入官方市场，获得曝光和分发渠道。

技术亮点：插件命名采用"不可变 slug"设计，避免重命名导致用户安装失效，体现了对已发布 API 兼容性的重视；`strict: false` 的 skill-bundle 模式允许没有 `plugin.json` 清单的仓库直接声明技能路径，且支持跨多级子目录的精细暴露，降低了技能类插件的接入门槛；`renames` 映射在用户下次同步时自动重写旧标识，实现了无感迁移。

上手建议：使用者可直接在 Claude Code 中运行 `/plugin install {plugin-name}@claude-plugins-official` 安装所需插件，或通过 Discover 界面浏览。贡献者应先阅读 `/plugins/example-plugin` 参考实现，内部插件由 Anthropic 团队开发，外部插件需通过官方提交表单申请；开发时需遵循标准目录结构，特别注意 `plugin.json` 元数据的正确性以及命名 slug 的不可变性。

#3 Python 连续 2 天

scientific-agent-skills

Turn any AI agent into an AI Scientist. The #1 Agent Skills library for science, used by 250,000+ scientists worldwide. 177 ready-to-use validated skills plus 100+ scientific databases covering biology, chemistry, medicine, and drug discovery. Compatible with Cursor, Claude Code, Codex, Pi, Antigravity, and the open Agent Skills standard.

Stars **50,090** Forks **4,464** Today **+1,091**

https://github.com/K-Dense-AI/scientific-agent-skills

项目简介：Scientific Agent Skills 是一个面向科研场景的"技能库"，为 AI Agent 提供 177 个经过验证的科学工作流技能，覆盖生物、化学、医学、药物发现等领域。它解决的核心问题是：通用 AI 助手缺乏科研领域的专业知识和规范，而该项目通过标准化技能包让任意 AI Agent 快速具备"AI 科学家"能力。

核心功能： - **177 个即用型科学技能**：每个技能封装了特定软件包、数据源或工作流的使用方法，包含科学惯例和验证检查。 - **100+ 科学数据库接入**：覆盖生物学、化学、医学、药物发现等领域的专业数据源。 - **跨 Agent 兼容**：遵循开放的 Agent Skills 标准，支持 Cursor、Claude Code、Codex、Google Antigravity 等主流宿主。 - **可移植插件包**：以 `plugin.json + skills/` 形式打包，支持 Agent Plugins 标准的客户端可一键加载整套技能。 - **配套 BYOK 工具**：K-Dense BYOK 提供本地化 AI 协科学家，支持自带 API Key、40+ 模型选择，数据留在本地。

适用场景：主要面向科研人员、生物信息学家、药物研发团队和需要处理科学数据的工程师。典型使用场景包括：用 AI Agent 分析实验数据、查询专业科学数据库、执行标准化科研工作流、撰写科学文档，以及在本地私有环境中进行敏感数据处理。

技术亮点：项目采用"程序性知识库"的设计理念——不是简单堆砌提示词，而是将每个科学工作流的结构化知识、约定和验证逻辑封装为可复用技能。arXiv 论文（2609.00065）对其方法论进行了学术化描述，同时项目配备安全扫描和技能测试 CI 流水线，保证技能质量。Python 实现加上开放标准兼容，使其具备良好的可扩展性和生态互操作性。

上手建议：普通用户可直接在 Cursor 或 Claude Code 中引用该技能库，或下载 K-Dense BYOK 桌面工具快速体验；建议先浏览 `docs/skill-guides/` 中的图解指南和 `docs/skills.md` 完整目录，挑选

与自身研究相关的技能。贡献者可从修复现有技能的验证逻辑或新增小众数据库接入入手，遵循 Agent Skills 标准提交 PR，并确保通过安全扫描和技能测试。

#4 Go NEW

caveman

🪨 why use many token when few token do trick. Viral skill + proxy for coding agents that cuts 65% of tokens by talking like a caveman.

Stars **112,198** Forks **6,506** Today **+398**

https://github.com/JuliusBrussee/caveman

项目简介：Caveman 是一个面向 AI 编码代理（coding agents）的"省 token"工具，核心理念是"能少说就少说"——让 AI 用极简的"穴居人式"表达回答问题和处理日志，从而大幅削减 token 消耗。它通过一个 skill + 代理层，在不牺牲代码准确性的前提下，把 AI 代理的输入输出都"瘦身"，号称能省下约 65% 的 token。

核心功能： 1. **Say less（说得少）**：让代理用极简语言回答（如把 63 token 的 React 解释压缩到 20 token），并提供 `/caveman`、`/ultracave`、`/megacave` 三档递减的压缩强度。 2. **Read less（读得少）**：在模型读取日志、JSON、CSV、测试输出前先做压缩，实测每次会话减少 33.2% 的输入 token。 3. **Auto（自动选择）**：根据任务自动挑选合适的模型和推理强度，在同等得分下比 Opus 高强度模式便宜约 54%。 4. **Blocks（代码模式）**：代理会保留自己写过的脚本并复用，避免重复生成，例如一次测试运行从 397 token 降到 76。 5. **Waste fixes（浪费修复）**：定位 token 消耗的"漏点"并针对性优化。

适用场景：主要面向重度使用 AI 编码代理（如 Claude Code、Codex、Gemini CLI 等 30+ 种代理）的开发者，尤其是那些按 token 计费、对成本敏感、或频繁让 AI 分析冗长日志/测试输出的团队。也适合个人开发者想在不降低代码质量的前提下压缩 AI 使用成本。

技术亮点：用 Go 编写，通过 npm CLI 一键安装并自动探测本机代理和登录态；它不是简单地截断输出，而是区分"自然语言解释"和"代码本身"——代码保持精确，只压缩冗余表达。项目有 Adobe Research 论文引用、JetBrains A/B 测试背书，并在 Hacker News 和 GitHub Trending 登顶，说明其 token 压缩效果经过了第三方验证，而非营销噱头。

上手建议：最快的方式是运行 `npx @caveman-ai/cli@latest`，它会自动找到你的代理并展示将要修改的内容，回车即完成配置。若只想体验"说得少"这一项，可单独安装对应 skill。想深入或贡献代码，建议先阅读 README 中的 Proof 和 Six rocks 章节理解各模块的度量方式，再从 `INSTALL.md` 支

持的代理列表和具体 rock 的开关机制入手，关注其 token 计量（tiktoken o200k）和压缩逻辑的实现。

#5 Python 连续 3 天

VoiceStudio

VoiceStudio is the open-source, fully-local ElevenLabs alternative — voice cloning, voice design, video dubbing, dictation, transcription & audiobook creation in 646 languages.

Stars **68,158** Forks **7,459** Today **+3,027**

https://github.com/debpalash/VoiceStudio

项目简介：VoiceStudio 是一个完全本地运行的开源语音 AI 工作台，定位为 ElevenLabs 的替代方案。它把语音克隆、配音、听写、转录、有声书制作等能力整合进一个桌面应用（Electron 构建），支持 646 种语言，所有处理都在用户自己的硬件上完成，无需上传数据到云端。

核心功能：一是语音克隆与语音设计，用户可上传参考音频克隆音色，或用文字描述生成全新声音；二是视频配音，支持带时间轴的对齐语音输出；三是听写与转录，配有悬浮小组件方便随时调用；四是有声书与批量任务制作，支持"配方（recipes）"和监听文件夹实现自动化流程；五是本地 API 与 MCP 接口，可让 AI Agent 直接调用语音能力。

适用场景：适合内容创作者（做视频配音、有声书）、播客与自媒体从业者、需要多语言本地化的团队，以及对数据隐私敏感、不愿把音频交给云端服务的用户。对开发者和 AI Agent 构建者来说，本地 API 和 MCP 也让它成为可集成的语音基础设施。

技术亮点：默认引擎基于 k2-fsa/OmniVoice，同时支持切换多种引擎；跨平台硬件适配做得较细，NVIDIA 走 CUDA、Apple Silicon 走 Metal（MPS）、无独显也能纯 CPU 运行，Windows on ARM 处于实验阶段；安装脚本会保留用户的设置、项目和模型，且明确不回退到旧的 Tauri 构建，工程规范上比较克制。

上手建议：macOS/Linux 用户可直接用一行 curl 脚本安装，Windows 用 PowerShell 的 irm 命令，或从 Releases 下载对应安装包（.dmg/.exe/.AppImage/.deb）。首次使用建议从"语音克隆"工作区入手，用一段干净的参考录音体验完整流程；想贡献代码的话，先读 docs/feature-catalog.md 了解引擎架构，再从文档或 Discord 社区找切入点。

#6 Shell NEW

superpowers

An agentic skills framework & software development methodology that works.

Stars **299,579** Forks **26,759** Today **+534**

https://github.com/obra/superpowers

项目简介：Superpowers 是一套面向 AI 编码代理（coding agent）的“软件开发方法论 + 技能框架”，由 Jesse Vincent（obra）打造。它解决的核心问题是：让 AI 代理不再一上来就乱写代码，而是先理解需求、梳理规格、制定计划，再按纪律执行。项目本质是一组可组合的“skills”加上引导指令，让代理自动触发这套流程。

核心功能： - **需求澄清与规格化**：代理启动后先反问用户“你到底想做什么”，把模糊需求逼成一份可阅读、可确认的规格文档，并分段展示供人审阅。 - **实现计划生成**：产出一份“连水平很差的初级工程师都能照着做”的详细计划，强调红绿 TDD、YAGNI、DRY 等工程原则。 - **子代理驱动开发（subagent-driven development）**：用户说“go”后，代理拆分任务、逐项执行、自查自审，可连续自主工作数小时而不偏离计划。 - **自动触发、零配置**：skills 会在合适时机自动激活，用户无需手动调用，代理“天然”具备这套能力。 - **多平台适配**：通过插件/扩展形式接入 Claude Code、Codex、Cursor、Gemini CLI、Copilot CLI、Devin、Factory Droid 等近二十种主流编码代理。

适用场景：适合重度使用 AI 编码代理的开发者、团队，尤其是希望让代理承担较长周期、较复杂功能开发，而不是只做零散补全的人。也适合企业团队在规范化的软件交付流程中引入 AI 代理协作。

技术亮点：项目主体用 Shell 编写，说明它更像是一套“技能包 + 安装脚本 + 提示词工程”的组合，而非传统库。其真正价值不在代码复杂度，而在于把软件工程方法论（TDD、YAGNI、规格先行、子代理分工）结构化为代理可执行的技能，并通过 marketplace/plugin 机制跨平台分发，实现“一次编写、多代理复用”。

上手建议：先根据你使用的代理（如 Claude Code 用 `/plugin install superpowers@claude-plugins-official`）完成安装，然后直接开一个新任务，观察代理是否主动进入“澄清需求→出规格→做计划→执行”的流程。想深入或贡献，建议从阅读 README 的 “What's Inside” 和 “Philosophy” 两节入手，理解 skills 的组织方式，再尝试新增或改进某个 skill。

#7 TypeScript NEW

ruflo

🌊 The original agent harness. Deploy intelligent multi-player swarms, coordinate autonomous workflows, and build conversational AI systems. Features adaptive memory, self-learning intelligence, federation, vector RAG integration, and native Claude Code / Codex / Hermes and many more Integrated

Stars **74,851** Forks **8,900** Today **+224**

https://github.com/ruvnet/ruflo

项目简介：Ruflo 是一个面向 Claude Code 和 Codex 的"智能体元框架"（agent meta-harness），核心理念是"Agent = 模型 + 框架"。它不训练模型，而是在模型外层提供工具、记忆、循环、沙箱和控制机制，让 AI 代理从"能跑"升级为"能协作"。项目原名 claude-flow，现以 ruvnet/ruflo 发布，已积累约 7.5 万星标。

核心功能： - **多智能体蜂群（Swarm）**：支持多个专业化代理自组织成团队，协同完成复杂任务。 - **自适应记忆与自学习**：内置持久化上下文和反馈学习机制，让代理能记住并改进。 - **联邦通信与向量 RAG 集成**：支持跨机器的代理通信，并集成向量检索增强上下文。 - **多客户端接入**：原生支持 Claude Code 插件、Codex，以及通过 MCP 协议接入其他工具。 - **可视化控制台与终端 UI**：提供 agent cockpit 和独立 TUI，可监控成本、事件、任务等。

适用场景：适合使用 Claude Code / Codex 进行 AI 辅助开发的工程师，尤其是需要多代理协作、长任务编排、跨会话记忆的复杂项目。也适合想构建对话式 AI 系统或需要企业级安全护栏的团队，以及研究多智能体架构的开发者。

技术亮点：以 MCP（Model Context Protocol）作为标准接入层，使框架与客户端解耦；采用插件化设计（core/console/mods 分离），mods 可在用户权限下执行路由与策略；提供独立的矢量记忆组件 RuVector，将本地向量搜索、图关系和反馈学习结合，形成"代理记忆"基础设施。

上手建议：在 Claude Code 中依次执行 `/plugin marketplace add ruvnet/ruflo` 及三个 plugin 安装命令即可体验；非 Claude Code 用户可运行 `npx ruflo@latest init wizard` 初始化项目。想快速了解全貌，建议先读官方 14 章指南《RuFlo Explained》；想贡献代码，可从 TypeScript 源码和插件模块入手，注意审查 mods 的权限行为。

#8 TypeScript 连续 4 天

context-mode

Context window optimization for AI coding agents. Sandboxes tool output (98% reduction), persists session memory, and enforces routing across 17 platforms via MCP + hooks.

Stars **29,635** Forks **2,065** Today **+902**

https://github.com/mksglu/context-mode

项目简介：Context Mode 是一个面向 AI 编程代理的 MCP 服务器，专门解决上下文窗口被工具输出快速耗尽的问题。它通过沙箱化工具输出、持久化会话记忆和跨平台路由，让代理在长时间编码任务中保持"记得住、想得清、不跑偏"。

核心功能： - **上下文沙箱**：将 Playwright 快照、GitHub issue、日志等原始工具输出拦截在上下文窗口之外，宣称可实现 98% 的体积压缩（315 KB → 5.4 KB）。 - **会话持久化**：把文件编辑、git 操作、任务状态、错误和用户决策写入 SQLite，压缩对话后通过 FTS5 + BM25 检索按需召回，而非全量回灌。 - **跨平台路由**：通过 MCP + hooks 在 17 个平台上统一调度，兼容 Cursor、Claude Code 等主流代理环境。 - **Think in Code**：引导 LLM 用代码而非自然语言进行推理，减少填充词和冗长解释带来的输出 token 浪费。

适用场景：适合重度使用 AI 编程代理的开发者与团队，尤其是需要长时间、多轮次、跨文件协作的复杂编码任务；也适合在 MCP 生态中构建代理工具链的工程师，用于降低 token 成本和上下文丢失率。

技术亮点：用 SQLite + FTS5 做会话事件的索引与检索，把"记忆"从上下文窗口外置到本地数据库，是架构上比较务实的选择；沙箱机制从源头拦截工具输出而非事后压缩，思路更彻底；`--continue` 与"新会话即清空"的语义设计，避免了旧数据污染新任务。

上手建议：从 npm 安装 `context-mode` 并按 README 配置 MCP 服务器即可接入现有代理；建议先在一个小项目上验证沙箱压缩和会话恢复效果，再逐步接入团队工作流。想贡献代码可从 TypeScript 源码中的平台适配层和 FTS5 检索逻辑入手，同时注意项目采用 ELv2 许可证，商用前需确认合规边界。

#9 Java NEW

ghidra

Ghidra is a software reverse engineering (SRE) framework

Stars **88,292** Forks **9,845** Today **+1,366**

https://github.com/NationalSecurityAgency/ghidra

项目简介：Ghidra 是由美国国家安全局（NSA）研究部门创建并维护的软件逆向工程（SRE）框架，旨在解决复杂逆向工程中的规模化与团队协作难题。它让安全分析师能够在 Windows、macOS 和 Linux 上分析编译后的二进制代码，是一个可定制、可扩展的高端研究平台。

核心功能：提供反汇编、汇编、反编译、图形化展示和脚本编写等全套分析工具；支持多种处理器指令集与可执行文件格式；既可在交互模式下使用，也能以自动化模式运行；用户可用 Java 或 Python 开发自定义扩展组件和脚本。

适用场景：主要面向安全研究员、恶意代码分析师和漏洞挖掘人员，用于分析恶意软件、理解闭源二进制程序逻辑、寻找网络与系统中的潜在漏洞，也适合团队协作式的大规模逆向工程项目。

技术亮点：作为 NSA 亲自打造并开源的重量级工具，它内置反编译器，能直接将二进制还原为接近 C 的伪代码；基于 Java 构建，跨平台且具备强大的插件与脚本扩展生态；其设计初衷就是应对真实情报场景中的规模化与团队协作需求。

上手建议：普通用户只需安装 JDK 25，下载官方多平台发布包解压后运行 `ghidraRun` 即可（也可用 PyGhidra）；开发者若想贡献代码，建议先阅读 `Developer's Guide`，用 Gradle 完成构建，并通过 Eclipse 的 GhidraDev 插件或 VS Code 集成来开发脚本与扩展。

#10 JavaScript NEW

ECC

The agent harness performance optimization system. Skills, instincts, memory, security, and research-first development for Claude Code, Codex, Opencode, Cursor and beyond.

Stars **279,973** Forks **41,648** Today **+1,015**

https://github.com/affaan-m/ECC

项目简介：ECC 是一个面向 AI 编程代理（Agent Harness）的性能优化系统，定位为"代理运行时的操作系统"。它试图解决当前 Claude Code、Codex、Cursor 等 AI 编程工具各自为政、缺乏统一能力层的问题，通过提供技能、直觉、记忆、安全和研究优先开发等模块，让不同代理框架都能获得一致的增强体验。

核心功能：一是**技能与直觉系统**，为代理注入可复用的领域知识和快速决策能力；二是**记忆管理**，让代理在跨会话、跨任务中保持上下文连续性；三是**安全层（AgentShield）**，对代理行为进行防护与审计；四是**研究优先的开发流程**，强调代理在动手前先做信息检索与验证；五是**多平台适配**，通过 ecc@ecc 插件统一接入 Claude Code、Codex、Opencode、Cursor 等宿主。

适用场景：主要面向重度使用 AI 编程代理的开发者、AI 工具链团队，以及希望把代理能力产品化、标准化的工程组织。典型场景包括：让 Claude Code 记住项目规范并跨会话复用、为多个代理统一配置安全策略、以及在复杂代码库中让代理先研究再改代码。

技术亮点：项目以 JavaScript 为主，但 README 显示其生态横跨 Shell、TypeScript、Python、Go、Java、Perl，说明它更像一个**多语言工具链的编排层**而非单一应用。通过 npm 包（ecc-universal、ecc-agentshield）、GitHub App、插件 slug 三种分发方式并存，形成"一次配置、多端生效"的架构。README 顶部用醒目的 WARNING 强调官方渠道，也反映出这类代理增强工具容易被仿冒、供应链风险较高的现实。

上手建议：最稳妥的路径是从官方 Claude Code 引导安装开始，使用 ecc@ecc 插件，避免手动叠加多种安装方式导致冲突。想贡献代码的话，建议先阅读多语言 README（含简体中文版）了解设计意图，再通过 Discord 社区沟通方向；同时务必只从官方 GitHub 仓库、npm 官方包和 ecc.tools 网站获取资源，警惕第三方镜像。

数据来源：github.com/trending

报告日期：2026-10-15

由 DeepSeek AI 提供智能分析 | 自动生成